

BRIEF DLA AI — Uniwersalny system sabotaży

Cel

Stworzyć uniwersalny system sabotaży, który będzie można podpiąć do różnych elementów gry:

- transport drogowy,
- odwiert,
- hub,
- rurociąg,
- magazyn,
- port,
- terminal,
- przyszłe elementy infrastruktury.

System ma działać podobnie architektonicznie do modułu ochrony i łapówek: ma mieć własny serwis, konfigurację w bazie, panel admina, logi oraz możliwość łatwego dodawania nowych typów sabotażu i efektów.

Na start wdramy tylko fundament oraz pierwsze podpięcie pod transport drogowy albo rurociąg, w zależności od tego, co jest łatwiejsze w obecnym kodzie.

1. Główna zasada

Nie kodować sabotaży na sztywno w PHP.

Nie robić osobnych mechanik typu:

- sabotaż odwiertu osobno,
- sabotaż huba osobno,
- sabotaż rurociągu osobno,
- sabotaż transportu osobno.

Zamiast tego stworzyć jeden uniwersalny system.

Typy sabotażu i ich efekty mają być definiowane w bazie danych oraz edytowane z panelu admina.

Admin ma móc dodać nowy typ sabotażu bez zmieniania kodu.

2. Struktura systemu

System składa się z:

- `SabotageService` — główny silnik sabotaży,
- `sabotage_options` — definicje typów sabotażu,
- `sabotage_effects` — efekty przypisane do sabotażu,

- `sabotage_attempts` — wykonane próby sabotażu,
 - `sabotage_logs` — historia sabotaży,
 - integracja z `ProtectionService`,
 - integracja z powiadomieniami,
 - integracja z finansami,
 - integracja z wiarygodnością firmy w przyszłości.
-

3. SabotageService

Dodać serwis:

```
src/SabotageService.php
```

albo:

```
src/Security/SabotageService.php
```

Serwis odpowiada za:

- pobranie dostępnych typów sabotażu,
 - wyliczenie kosztu sabotażu, jeśli sabotaż jest akcją gracza,
 - wyliczenie ryzyka powodzenia,
 - wykonanie próby sabotażu,
 - zapis skutków,
 - zastosowanie efektów,
 - zapis logów,
 - wysłanie powiadomień,
 - sprawdzenie aktywnej ochrony celu,
 - zmniejszenie skutków sabotażu, jeśli cel ma ochronę.
-

4. Dwa typy sabotażu

System powinien obsługiwać dwa modele.

4.1. Sabotaż systemowy

To sabotaż generowany przez grę.

Przykłady:

- atak na transport w regionie wysokiego ryzyka,
- uszkodzenie rurociągu przez grupę przestępczą,
- kradzież części z odwiertu,
- celowe zatrzymanie pracy huba,
- atak na magazyn.

Ten typ działa jako zdarzenie ryzyka w ticku.

4.2. Sabotaż gracz kontra gracz — TODO

To opcja na później.

Gracz może próbować zaszkodzić innemu graczowi.

Nie wdrażać tego w pierwszym etapie, bo wymaga balansu, zabezpieczeń i anti-abuse.

Na start tylko zapisać jako TODO.

5. Tabela sabotage_options

Tabela przechowuje definicje typów sabotażu.

Przykładowe pola:

```
id
code
name
description
target_type
context
is_active
base_chance_pct
cost_type
cost_value
cost_currency
severity
cooldown_minutes
min_region_risk
requires_black_market
sort_order
created_at
updated_at
```

Znaczenie pól

`code`

Techniczny kod sabotażu, np. `road_ambush`, `pipeline_damage`, `well_equipment_theft`.

`name`

Nazwa widoczna w adminie albo w logach.

`description`

Opis sabotażu prostym językiem.

`target_type`

Typ celu.

Przykłady:

```
road_transport
well
hub
pipeline
warehouse
port
terminal
```

`context`

Kontekst użycia.

Przykłady:

```
road_transport_sabotage
well_sabotage
hub_sabotage
pipeline_sabotage
```

`base_chance_pct`

Bazowa szansa powodzenia sabotażu.

`cost_type`

Sposób liczenia kosztu, jeśli sabotaż jest akcją gracza.

Przykłady:

```
fixed
percent_reference
per_bbl
```

`cost_value`

Wartość kosztu.

`cost_currency`

Źródło płatności.

Przykłady:

```
cash
bank
black_market
```

severity

Poziom ciężkości sabotażu.

Przykłady:

```
low
medium
high
critical
```

min_region_risk

Minimalne ryzyko regionu, jeśli sabotaż ma występować tylko w trudniejszych regionach.

requires_black_market

Czy typ sabotażu wymaga czarnego rynku.

6. Tabela sabotage_effects

Efekty mają być osobno, żeby można było dodawać nowe skutki bez zmiany kodu.

Przykładowe pola:

```
id
sabotage_option_id
effect_key
effect_type
effect_value
created_at
updated_at
```

Przykładowe effect_key

```
oil_loss_pct
oil_loss_fixed
delay_minutes
damage_pct
condition_loss
capacity_loss_pct
production_stop_minutes
transport_loss_pct
repair_cost_pct
```

```
hub_buffer_loss_pct  
pipeline_flow_loss_pct  
security_bypass_chance  
company_credibility_delta  
black_market_score_delta  
legal_case_risk_pct
```

7. Przykłady sabotaży

7.1. Napad na transport

```
target_type = road_transport
```

Efekty:

```
transport_loss_pct = 100  
delay_minutes = 0
```

Opis:

Transport został przejęty. Cały kurs zostaje utracony.

Ważne: jeśli nastąpi kradzież albo napad, to towar nie może zostać dostarczony. Nie może być tak, że skradziono część, a reszta magicznie dojechała, jeśli typ zdarzenia oznacza przejęcie transportu.

7.2. Częściowa kradzież transportu

```
target_type = road_transport
```

Efekty:

```
transport_loss_pct = 30  
delay_minutes = 15
```

Opis:

Część transportu została utracona, a kurs dociera opóźniony.

Ten typ jest osobny od pełnego napadu.

7.3. Uszkodzenie rurociągu

```
target_type = pipeline
```

Efekty:

```
condition_loss = 20  
pipeline_flow_loss_pct = 50  
repair_cost_pct = 8
```

Opis:

Rurociąg został uszkodzony. Przepływ spada do czasu naprawy.

7.4. Sabotaż odwiertu

```
target_type = well
```

Efekty:

```
condition_loss = 15  
production_stop_minutes = 30  
repair_cost_pct = 5
```

Opis:

Odwiert zostaje czasowo zatrzymany i wymaga naprawy.

7.5. Kradzież sprzętu z odwiertu

```
target_type = well
```

Efekty:

```
production_stop_minutes = 20  
repair_cost_pct = 3
```

Opis:

Zginęła część sprzętu technicznego. Produkcja zostaje czasowo ograniczona.

7.6. Sabotaż huba

```
target_type = hub
```

Efekty:

```
condition_loss = 10
capacity_loss_pct = 35
hub_buffer_loss_pct = 20
```

Opis:

Hub traci część przepustowości i może utracić część ropy z bufora.

8. Tabela sabotage_attempts

Tabela zapisuje wykonane próby sabotażu.

Przykładowe pola:

```
id
player_id
source_type
source_id
target_player_id
target_type
target_id
sabotage_option_id
context
status
chance_pct
roll_value
cost
paid_from
detected
protection_applied
created_at
resolved_at
meta_json
```

Statusy:

```
success
failed
blocked_by_protection
partially_blocked
detected
cancelled
```


Wyjaśnienie

`player_id`

Gracz wykonujący sabotaż albo `NULL`, jeśli sabotaż jest systemowy.

`target_player_id`

Właściciel celu.

`target_type`

Typ celu, np. transport, odwiert, hub.

`target_id`

ID celu.

`detected`

Czy wykryto sprawcę albo źródło sabotażu.

`protection_applied`

Czy ochrona celu wpłynęła na wynik.

9. Tabela sabotage_logs

Historia sabotaży.

Pola:

```
id
sabotage_attempt_id
player_id
target_player_id
target_type
target_id
event_key
message
meta_json
created_at
```

Przykładowe `event_key`:

```
sabotage_attempted
sabotage_success
sabotage_failed
sabotage_blocked_by_protection
sabotage_partially_blocked
sabotage_detected
sabotage_effect_applied
```

10. Integracja z ProtectionService

Sabotaż musi sprawdzać aktywną ochronę celu.

Przykład:

1. SabotageService losuje lub uruchamia próbę sabotażu.
2. System pobiera aktywne efekty ochrony:

```
ProtectionService::getActiveEffects($playerId, $targetType, $targetId, $context)
```

1. Efekty ochrony zmniejszają szansę powodzenia albo skutki sabotażu.
2. Sabotaż może zostać:
3. zablokowany,
4. częściowo ograniczony,
5. wykonany mimo ochrony.

Ważne

Ochrona nie daje 100% bezpieczeństwa.

Ochrona ma zmniejszać ryzyko i skutki.

11. Metody SabotageService

getAvailableOptions

```
getAvailableOptions(string $targetType, string $context): array
```

Zwraca dostępne typy sabotażu dla celu.

quote

```
quote(int $playerId, string $optionCode, float $referenceValue, string $targetType, int $targetId): array
```

Do użycia później przy sabotażu gracz kontra gracz.

Na P1 może być nieużywane.

attempt

```
attempt(  
    ?int $playerId,  
    int $targetPlayerId,  
    string $optionCode,  
    string $targetType,  
    int $targetId,  
    string $context,  
    float $referenceValue = 0,  
    array $meta = []  
): array
```

Wykonuje próbę sabotażu.

Zwraca:

```
[  
    'success' => true,  
    'outcome' => 'success',  
    'detected' => false,  
    'protection_applied' => true,  
    'message' => 'Sabotaż zakończony sukcesem.'  
]
```

applyEffects

```
applyEffects(int $attemptId): void
```

Nakłada skutki sabotażu na cel.

processSystemSabotage

```
processSystemSabotage(int $playerId): void
```

Metoda dla ticka lub serwisu zdarzeń.

Nie dopisywać dużej logiki bezpośrednio do ticka.

Tick może tylko wywołać serwis.

12. Podpięcie P1

Na start wdrożyć tylko jeden lub dwa obszary.

Rekomendacja:

P1A — transport drogowy

Sabotaż transportu drogowego może powodować:

- pełną utratę kursu,
- częściową utratę kursu,
- opóźnienie,
- dodatkowy koszt,
- powiadomienie gracza.

Ochrona transportu może zmniejszać ryzyko sabotażu.

P1B — rurociąg

Sabotaż rurociągu może powodować:

- spadek kondycji,
- spadek przepustowości,
- konieczność naprawy,
- zatrzymanie przepływu.

Jeśli wdrożenie dwóch obszarów jest zbyt duże, zacząć tylko od transportu drogowego.

13. Komunikaty dla gracza

Sabotaż transportu

Transport został zaatakowany

Część kursu została utracona. Sprawdź szczegóły w logistyce.

Pełna utrata transportu

Transport został przejęty

Cały kurs został utracony. Towar nie dotarł do celu.

Sabotaż mimo ochrony

Sabotaż mimo ochrony

Ochrona ograniczyła ryzyko, ale nie wyeliminowała go całkowicie.

Ochrona zablokowała sabotaż

Ochrona zadziałała

Próba sabotażu została zatrzymana przez ochronę.

Sabotaż rurociągu

Rurociąg został uszkodzony

Przepływ został ograniczony. Wymagana jest naprawa.

14. Panel admina

Dodać panel:

Admin → Sabotaże

Zakładki:

Typy sabotażu

Admin może:

- dodać typ sabotażu,
- edytować nazwę,
- edytować opis,
- wybrać typ celu,
- wybrać kontekst,
- ustawić bazową szansę,
- ustawić ciężkość,
- włączyć lub wyłączyć typ.

Efekty sabotażu

Admin może przypisywać efekty do typu sabotażu.

Próby sabotażu

Admin widzi:

- kiedy wystąpił sabotaż,
- jaki był cel,
- jaki był wynik,
- czy ochrona zadziałała,
- czy skutki zostały ograniczone.

Logi sabotażu

Admin widzi historię zdarzeń.

15. Balans

Sabotaż nie może być zbyt częsty.

Jeśli będzie zbyt częsty, gracz będzie czuł, że gra go karze.

Zasada:

- małe sabotaże mogą występować częściej,
 - duże sabotaże rzadziej,
 - region wysokiego ryzyka może zwiększać szansę,
 - ochrona zmniejsza ryzyko,
 - bardzo ciężkie skutki powinny być rzadkie.
-

16. Czego nie wdrażać teraz

Nie wdrażać teraz:

- sabotażu gracz kontra gracz,
- prywatnych armii,
- wojny korporacyjnej,
- śledztw,
- sądów,
- pełnych kar prawnych,
- automatycznych odwetów,
- sabotażu portów i terminali,
- sabotażu wszystkich systemów naraz.

Na start:

silnik + admin + transport drogowy lub rurociąg.

17. Najkrótsza wersja dla AI

Stworzyć uniwersalny system sabotaży oparty o `SabotageService`, `sabotage_options`, `sabotage_effects`, `sabotage_attempts` i `sabotage_logs`. Typy sabotażu i efekty mają być konfigurowalne w panelu admina. System ma współpracować z `ProtectionService`, czyli ochrona może zmniejszać ryzyko albo skutki sabotażu. Na start podpiąć transport drogowy, ewentualnie rurociąg. Nie wdrażać jeszcze sabotażu gracz kontra gracz.